

Course on Knowledge Graphs in the era of Large Language Models

Session 1: Creating a small KG

Ernesto Jiménez-Ruiz

February 6, 2026

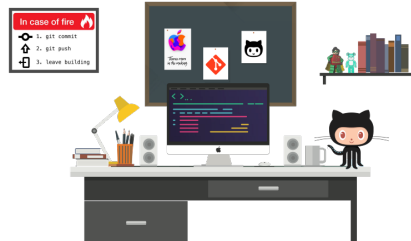
Contents

1	Git Repository	2
2	Infrastructure set-up	2
2.1	Accessing the example codes	2
2.2	Getting around with Git	2
2.3	Python libraries	3
2.3.1	Key libraries	4
2.4	Testing the infrastructure set-up	5
3	Creating RDF triples	5
4	Protégé ontology editor	6
5	FOAF - Friend of a friend (Optional)	6
6	Solutions	7

1 Git Repository

Support codes for the laboratory sessions are available in *github*.

<https://github.com/city-knowledge-graphs/kg-course-valgrai>



2 Infrastructure set-up

2.1 Accessing the example codes

In order to download the codes in the GitHub repository, you can (1) use Git and clone/fork the repository (see instructions in Section 2.2), or (2) directly download the full repository as a ZIP file. Option (1) will allow you to be up-to-date with my solutions and additional material without downloading the whole repository; option (2) is simpler, but you will need to download the whole repository every time there is a change.

2.2 Getting around with Git

If you decide to go with Option (2), you can ignore this section.

Create a GitHub account. If you do not have already a GitHub account, you can set up a free account by visiting <https://github.com/>. GitHub is a public commercial widely service owned by Microsoft. Gitlab is an alternative solution (<https://about.gitlab.com/>). You can create both public (like the ones in <https://github.com/city-knowledge-graphs>) and private repositories.

Git tutorials. The underlying technology for distributed version control/collaboration is called *git*. You can find several tutorial on the Web, for example:

- <https://www.w3schools.com/git/>
- https://www.youtube.com/watch?v=SWYqp7iY_Tc
- <https://www.freecodecamp.org/news/learn-the-basics-of-git-in-under-10-minutes-da548267cc91/>

Desktop application. Most tutorials refer to the command line for git. Using the command-line or terminal window is typically natural for macOS and Linux users, but may not be that common for Windows user. GitHub Desktop (<https://desкто>

p.github.com/) may be an easy solution to manage your git repositories. It is only available for macOS and Windows users. It comes with documentation (<https://docs.github.com/en/desktop>)

Command-line. Check if git is installed in your computer (*e.g.*, type in the command line `git --help`). You can download/install git from <https://git-scm.com/downloads>).

`clone` is the command to clone and download a git repository. First select the folder where you would like to download the codes and then run the command:

- `git clone https://github.com/city-knowledge-graphs/kg-course-valgrai.git`

To update the local copy with new remote changes (*e.g.*, codes of future laboratory sessions) use the command `pull`: `git pull` from the respective local repository folders. To avoid clashes with the changes on the main branch it is suggested to add generated codes and data in the folder `student-codes-data`.

2.3 Python libraries

We will use Python 3.10 (or newer). Check if you have python installed in your computer with (`python --version` or `python3 --version`). Otherwise install from <https://www.python.org/downloads/>

I use Eclipse as Python editor, but Spyder IDE (<https://www.spyder-ide.org/>) and Visual Studio (<https://visualstudio.microsoft.com/>) is typically the choice for many Python developers.

In order to deal with the required libraries and dependencies for each lab sessions it is recommended to use a virtual environment. I typically use `pip` (<https://pip.pypa.io/en/stable/installation/>) and `venv` (<https://docs.python.org/3/library/venv.html>), but there are alternatives like `conda` (<https://docs.conda.io/>). VS Code (<https://code.visualstudio.com/docs/python/environments>) and Spyder (<https://docs.spyder-ide.org/current/faq.html#using-spyder>) also allow the set-up of environments. Jupyter notebooks in VC code: <https://code.visualstudio.com/docs/datascience/jupyter-notebooks>

To set up a new environment:

```
python3 -m venv /home/ernesto/path_to_lab1_venv
```

To activate the environment, depends on the platform (<https://python.land/virtual-environments/virtualenv>). For example:

```
source /home/ernesto/path_to_lab1_venv/bin/activate
```

To deactivate a virtual environment: `deactivate`

The required libraries are in the GitHub repository: `requirements.txt` file. To install the libraries in the virtual environment:

```
pip3 install -r requirements.txt
```

Windows commands: using the command line in Windows is a bit different with respect to Linux and macOS. These are the typical commands:

- Creation of a virtual environment:

```
py -m venv C:\Users\ernesto\path_to_env
```

- Activation of virtual environment (execute "activate" script):

```
C:\Users\ernesto\path_to_env\Scripts\activate
```

- Installing pip and making sure pip, setuptools and wheels are up to date using:

```
py -m pip install --upgrade pip setuptools wheel
```

- Installing requirements (requirements file in GitHub). Run one of the following:

```
py -m install -r requirements.txt
```

```
py -m pip install -r requirements.txt
```

2.3.1 Key libraries

- Jupyter notebook: Web-based application to execute (python) code together with comments.

- Docs: <https://jupyter-notebook.readthedocs.io/en/latest/>

- Example in lab1: `jupyter notebook lab1-notebook.ipynb`

- RDFLib: a library to manage RDF-based knowledge graphs.

- Docs: <https://rdflib.readthedocs.io/en/stable/>

- Examples: <https://github.com/RDFLib/rdflib>

- SPARQLWrapper: a python wrapper around a SPARQL service.

- Docs and examples: <https://github.com/RDFLib/sparqlwrapper>

- Owlready: a package for ontology-oriented programming in Python.

- Docs: <https://owlready2.readthedocs.io/en/latest/intro.html>

- OWL-RL: A simple library to perform reasoning on the OWL2 RL Profile.

- Docs: <https://owl-rl.readthedocs.io/en/latest/>

2.4 Testing the infrastructure set-up

The codes provided in the folder *test* give an overview about some of the functionalities we will use in the course. They include methods for:

- Loading and ontology and printing its classes (*test-load-ontology.py* script/class).
- Loading and querying a local RDF knowledge graph (*test-load-RDFGraph.py* script).
- Querying a remote RDF knowledge graph via a SPARQL endpoint (*test-query-endpoint.py* script).
- Performing OWL reasoning within the RL profile (*test-owl-reasoning.py* script).
- Read tabular data from a CSV file (*test-open-csv.py* script).

Exercise 1.1: Check you can run the python scripts (or the `test-notebook.ipynb` Jupyter notebook), without compilation errors. At this stage you do not need to understand all the details, but you can start giving a look to the codes and get familiar with the libraries.

3 Creating RDF triples

RDFLib provide methods to populate an empty graph or add triples to an existing one.

RDFLib documentation:

- https://rdflib.readthedocs.io/en/stable/intro_to_creating_rdf.html
- https://rdflib.readthedocs.io/en/stable/rdf_terms.html

Task 1.1: Transform Table 1 into triples.

- Try to first create them with a text editor.
- Create the triples using RDFLib (programmatically). Tip: *An example script to create triples is given in GitHub.*
- Try to create triples by automatically processing the CSV file representing Table 1. Tip: *An example script to load a csv file is given in GitHub.*

Table 1: Table about companies

Company	Founding year	Headquarters
OST	2017	Oxford
DeepReason.ai	2018	Oxford
Oxstem	2011	Oxford
Oxbotica	2014	Oxford
DeepMind	2010	London

4 Protégé ontology editor

Protégé is an editor of OWL ontologies (<https://protege.stanford.edu/>). We will use the desktop version.

The pizza ontology and its tutorial are well-known resources in the Semantic Web community. They were developed for educational purposes by the University of Manchester. The tutorial have been recently updated by Michael DeBellis.

The pizza ontology and the tutorial are found at:

- <https://tinyurl.com/NewPizzaTutorialV3-2>
- <http://protege.stanford.edu/ontologies/pizza/pizza.owl>

Task 1.2: Install Protégé (Desktop version) and load the Pizza ontology (File → Open from URL: <https://protege.stanford.edu/ontologies/pizza/pizza.owl>).

Task 1.3: Take some time to explore the interface. You may want to check the first steps of the tutorial. Browse the class hierarchy, the property hierarchies and the individuals and note how the ontology describes the domain of pizzas. Check the documentation if necessary: <http://protegeproject.github.io/protege/>.

Task 1.4: Find *Margherita* and see how it is defined as a pizza with only cheese and tomato topping. Protégé uses OWL, and thus formal semantics to define the concepts of the domain. Look at the definition of *VegetarianPizza*. Is a *MargheritaPizza* a vegetarian pizza? Why / why not?

5 FOAF - Friend of a friend (Optional)

The FOAF project is an old project where RDF was the core technology. FOAF aimed at being a simple technology to make it easier to share and use information about people and their activities. See [https://en.wikipedia.org/wiki/FOAF_\(ontology\)](https://en.wikipedia.org/wiki/FOAF_(ontology)) for more information.

Task 2.7: (Optional) Go to the FOAF-a-Matic online service (<http://ldodds.com/foaf/foaf-a-matic.en.html>) and create your own FOAF file. Add myself as friend (See *Also* field; no strong commitments here :-)): <https://raw.githubusercontent.com>

`usercontent.com/city-knowledge-graphs/foaf/main/ernesto_foaf.rdf`.

Task 1.5: (Optional) Upload your generated FOAF RDF file to `https://github.com/city-knowledge-graphs/foaf` via a pull-request. Alternatively, send me the generated FOAF RDF file.

FOAF was an interesting project, but there are currently more recent efforts. For example, Wikidata (`https://www.wikidata.org/`) is a community driven knowledge graph including both manually and automatically created entries. Check my entry: `https://www.wikidata.org/wiki/Q56614973`.

6 Solutions

Task 1.1: The model solution assumes the existence of manual or automatic mapping of the entities in the CSV file (Table 1) to a KG like DBpedia. Instead of DBpedia one could use a custom ontology.

Manual annotation is time-consuming for very large datasets and KGs. There are however systems that try to (semi)automate the process (*e.g.*, systems participating in the SemTab challenge: `https://www.cs.ox.ac.uk/isg/challenges/sem-tab/`). Automatic systems typically find the semantic type of a column, the relation between columns, and correspondences between cells and entities in the KG. For example, for the Table 1:

- Elements in Column 0 are of type `dbo:Company`
- Elements of Column 2 are of type `dbo:City`
- Columns 0 and 1 are related via the predicate `dbo:foundingYear`
- Columns 0 and 2 are related via the predicate `dbo:headquarter`
- Some cells can also be matched to KG entities:
 - `dbr:Oxford`
 - `dbr:London`
 - `dbr:DeepMind`
 - `dbr:Oxbotica`

In the *Tabular Data to Knowledge Graphs* session, we will implement a basic system that finds candidate KG entities for a given cell using the DBpedia look-up service.

For the task in this lab, the model solution implements a very basic look-up relying on a very small dictionary with the above entities. A fresh URI is created for the cells without an external KG correspondence (*e.g.*, *OST*, *DeepReason.ai*, *Oxstem*).

The model solutions are available in the respective GitHub repositories. Python: `Solution_Task1_table.py` and `Solution_Task1_table.ipynb`.